

1. MLP Baseline

1.1 问题与任务

MNIST 含 60,000 张训练图与 10,000 张测试图 (28×28 灰度)。Part A 要求自行实现全连接层前向/反向、带 Softmax 的多分类交叉熵；训练 MLP 并汇报 train/val 与学习曲线。

1.2 实现说明

Part A 在 MNIST 上实现全连接前向/反向、带 Softmax 的多分类交叉熵，并训练 $784 \rightarrow 600$ (ReLU) $\rightarrow 10$ 的 MLP。核心算子位于 `codes/mynn/op.py`，模型封装在 `codes/mynn/models.py` (`Model_MLP`)，优化与学习率调度在 `codes/mynn/optimizer.py`、`codes/mynn/lr_scheduler.py`；训练与作图入口为 `codes/test_train.py`，测试集评估为 `codes/test_model.py`。

张量形状 (batch 大小记为 N ，输入已展平并归一化到 $[0, 1]$)：

阶段	形状
输入	$(N, 784)$
Linear(784→600)	$(N, 600)$
ReLU	$(N, 600)$
Linear(600→10)	$(N, 10)$
Softmax + 交叉熵	标量 loss；反向时对 logits 输出 $(N, 10)$ 梯度

可训练参数量约为 $784 \times 600 + 600 + 600 \times 10 + 10 \approx 4.77 \times 10^5$ ；展平表示未显式利用 28×28 的空间邻域结构。

(1) 全连接层 Linear

记 mini-batch 输入 $X \in \mathbb{R}^{N \times d_{\text{in}}}$ ，权重 $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ ，偏置 $b \in \mathbb{R}^{1 \times d_{\text{out}}}$ 。前向：

$$Z = XW + b$$

反向时设上游梯度为 $G = \partial \mathcal{L} / \partial Z \in \mathbb{R}^{N \times d_{\text{out}}}$ ，则

$$\frac{\partial \mathcal{L}}{\partial W} = X^\top G, \quad \frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N G_{i,:}, \quad \frac{\partial \mathcal{L}}{\partial X} = GW^\top$$

Model_MLP 对每层 Linear 使用 He 初始化 $W_{ij} \sim \mathcal{N}(0, \sqrt{2/\text{fan_in}})$ ，以匹配 ReLU 的方差传播。两层全连接在构造时开启 L2 (`weight_decay=True`， $\lambda = 10^{-4}$)，衰减在 `SGD.step()` 中与梯度更新同一步完成（见 (5)）。

Python 实现：

```
1 class Linear(Layer):
2     """
3     全连接层：前向  $Z = XW + b$ ；反向写 grads['W']、grads['b'] 并将 grad
    @ W.T 传回上一层。
4     """
5     def forward(self, X):
6         self.input = X
7         return X @ self.W + self.b
8
9     def backward(self, grad: np.ndarray):
10        # ——  $\partial \mathcal{L} / \partial W$ ,  $\partial \mathcal{L} / \partial b$ ,  $\partial \mathcal{L} / \partial X$  ——
11        self.grads['W'] = self.input.T @ grad
12        self.grads['b'] = np.sum(grad, axis=0, keepdims=True)
13        return grad @ self.W.T
```

(2) 激活层 ReLU

$$\text{ReLU}(x) = \max(0, x), \quad \frac{\partial \text{ReLU}(x)}{\partial x} = \mathbb{1}_{x>0}$$

ReLU 在负半轴梯度为 0，缓解 Sigmoid 类饱和；本网络仅在隐藏层使用一层 ReLU。该层无参数，`optimizable=False`，不参与 `SGD.step()`。

Python 实现:

```
1 class ReLU(Layer):
2     def forward(self, X):
3         self.input = X
4         return np.where(X < 0, 0, X)
5
6     def backward(self, grads):
7         return np.where(self.input < 0, 0, grads)
```

(3) Softmax 与多分类交叉熵 MultiCrossEntropyLoss

对 logits $z \in \mathbb{R}^C$ ($C = 10$)，数值稳定 Softmax:

$$p_k = \frac{\exp(z_k - \max_j z_j)}{\sum_j \exp(z_j - \max_j z_j)}$$

单样本损失 $-\log p_y$ (y 为真类)，batch 平均:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log(p_{y^{(i)}}^{(i)} + \varepsilon), \quad \varepsilon = 10^{-12}$$

将 Softmax 与交叉熵合并求导时，对 logits 的梯度为 $(p - \mathbf{e}_y)/N$ ，其中 $\mathbf{e}_y$ 为 one-hot。`backward()` 先算该梯度，再调用 `model.backward` 链式传至各 `Linear`。

Python 实现:

```

1 def softmax(X):
2     x_max = np.max(X, axis=1, keepdims=True)
3     x_exp = np.exp(X - x_max)
4     return x_exp / np.sum(x_exp, axis=1, keepdims=True)
5
6
7 class MultiCrossEntropyLoss(Layer):
8     def forward(self, predicts, labels):
9         self.predicts, self.labels = predicts, labels
10        prob = softmax(predicts) if self.has_softmax else predicts
11        batch_size = predicts.shape[0]
12        correct_prob = prob[np.arange(batch_size), labels]
13        return -np.mean(np.log(correct_prob + 1e-12))
14
15    def backward(self):
16        batch_size = self.predicts.shape[0]
17        prob = softmax(self.predicts)
18        self.grads = prob.copy()
19        self.grads[np.arange(batch_size), self.labels] -= 1
20        self.grads /= batch_size
21        self.model.backward(self.grads)

```

(4) 模型 Model_MLP 与前向/反向链

test_train.py 实例化 Model_MLP([784, 600, 10], 'ReLU', [1e-4, 1e-4]) : 相邻维度间插入 Linear + ReLU (最后一层仅 Linear) 。 forward 顺序调用 layers ; backward 自损失梯度起逆序调用各层 backward 。

Python 实现:

```

1 class Model_MLP(Layer):
2     def __init__(self, size_list=None, act_func=None,
3         lambda_list=None):
4         # —— 784→600→10: He 初始化 Linear, 前两层 λ=1e-4 ——
5         for i in range(len(size_list) - 1):
6             layer = Linear(in_dim=size_list[i],
7                 out_dim=size_list[i + 1], initialize_method=he_init)

```

```

6         if lambda_list is not None:
7             layer.weight_decay = True
8             layer.weight_decay_lambda = lambda_list[i]
9         self.layers.append(layer)
10        if i < len(size_list) - 2:
11            self.layers.append(ReLU())
12
13    def forward(self, X):
14        outputs = X
15        for layer in self.layers:
16            outputs = layer(outputs)
17        return outputs
18
19    def backward(self, loss_grad):
20        grads = loss_grad
21        for layer in reversed(self.layers):
22            grads = layer.backward(grads)
23        return grads

```

(5) 优化器 `SGD` 与 L2 权重衰减

记当前学习率为 α (即 `optimizer.init_lr` , 由 (6) 的调度器更新) 。对每个可训练参数 θ , 若该层开启 L2 , 先作权重衰减再梯度下降:

$$\theta \leftarrow \theta(1 - \alpha\lambda) - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

Part A 两层 `Linear` 的 λ 均为 10^{-4} 。 `RunnerM` 每个 batch 的顺序为: 前向 $\rightarrow$

`loss_fn` 求 loss $\rightarrow$ `loss_fn.backward()` $\rightarrow$ `optimizer.step()` $\rightarrow$

`scheduler.step()` ; 默认 `eval_each_iter=True` , 每处理一个训练 batch 即在 10,000

张验证集上评估一次, 用于选取 `best_models/best_model.pickle` 。

Python 实现:

```

1 class SGD(Optimizer):
2     def step(self):
3         for layer in self.model.layers:
4             if layer.optimizable:
5                 for key in layer.params.keys():
6                     if layer.weight_decay:
7                         layer.params[key] *= (1 -
self.init_lr * layer.weight_decay_lambda)
8                         layer.params[key] -= self.init_lr *
layer.grads[key]

```

(6) 学习率调度 MultiStepLR

在迭代步 k 属于里程碑集合 $\mathcal{M} = \{800, 2400, 4000\}$ 时，将学习率乘以 $\gamma = 0.5$ ，初值 $\alpha_0 = 0.06$ ，故 α_k 序列为 $0.06 \rightarrow 0.03 \rightarrow 0.015 \rightarrow 0.0075$ ($k \geq 4000$ 后保持不变)。调度器只修改 `optimizer.init_lr`，供下一步 SGD 使用。

$$\alpha_k := \begin{cases} \gamma \alpha_{k-1}, & k \in \mathcal{M} \\ \alpha_{k-1}, & \text{otherwise} \end{cases}$$

Python 实现:

```

1 class MultiStepLR(scheduler):
2     def step(self) -> None:
3         self.step_count += 1
4         if self.step_count in self.milestones:
5             self.optimizer.init_lr *= self.gamma

```

`test_train.py` 中的实例化: `SGD(init_lr=0.06, ...)` 与 `MultiStepLR(..., milestones=[800, 2400, 4000], gamma=0.5)`。

1.3 实验结果

实验设置：使用 MNIST 官方 60,000 张训练图（展平为 784 维）。

`codes/test_train.py` 在 `np.random.seed(309)` 下打乱索引，前 10,000 张作验证集、后 50,000 张作训练集，索引写入 `codes/idx.pickle`（Part B/C 复用同一划分）。像素除以训练集最大值归一化到 $[0, 1]$ 。训练由 `RunnerM` 完成：

`num_epochs=5`，`batch_size=32`；优化器与学习率调度见 §1.2 (5)(6)。

`eval_each_iter=True` 时**每个训练 batch 后**在验证集上评估，按验证准确率最高保存

`codes/best_models/best_model.pickle`（`log_iters=100` 仅控制日志打印）。官方 10,000 张测试集不参与选模，由 `codes/test_model.py` 评估；摘要与曲线见

`codes/results/part_a_summary.txt`、`part_a_learning_curve.png`。

在上述设置下，验证集与测试集结果如下。

指标	数值
最佳验证准确率	95.04%
测试集准确率	95.20%
末轮训练 loss / accuracy	0.0678 / 100.00%

末轮验证 loss / accuracy	0.1770 / 95.04%
----------------------	-----------------

来源：`codes/results/part_a_summary.txt`、
`codes/results/part_a_test_accuracy.txt`。

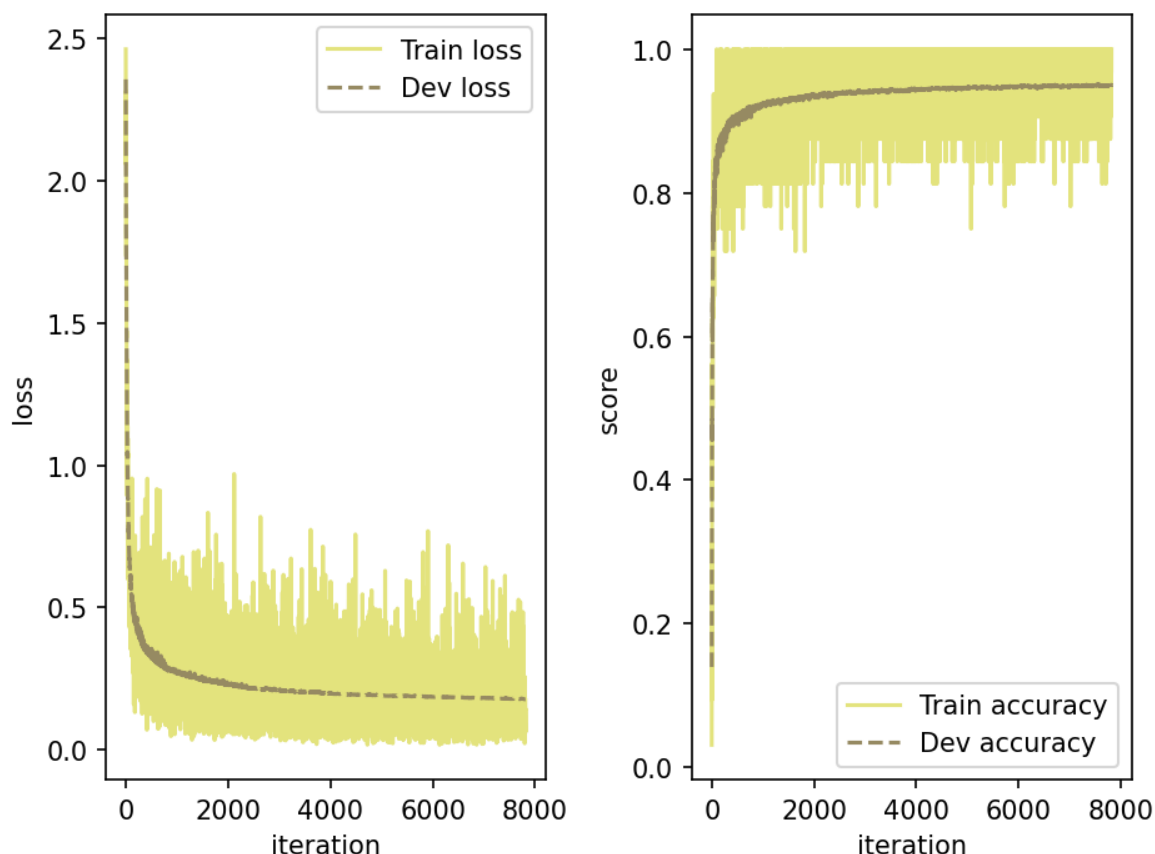


图 1: Part A MLP 学习曲线 (`codes/results/part_a_learning_curve.png`, 由 `test_train.py` 绘制)。横轴为 `iteration` (约 8,000 步, 对应 5 epoch、`batch_size=32` 且 `eval_each_iter=True` 时每 `batch` 记录一次 dev)。左: **loss** — 训练 loss (黄实线) `batch` 级波动大, 由约 2.5 快速降至 0.5 以下; 验证 loss (棕虚线) 平滑下降至约 0.18, 与 `part_a_summary.txt` 末轮 dev loss 0.1770 一致。右: **score** — 验证 accuracy 单调升至约 0.95 平台 (最佳 95.04%) ; 训练 accuracy 噪声大、常触及 1.0, 与末轮 train accuracy 100% 一致, 体现训练—验证间隙与轻度过拟合。里程碑 800 / 2400 / 4000 处学习率减半后, dev 曲线仍稳步改善而未发散。

2. CNN Model and MLP-vs-CNN Comparison

2.1 任务

实现 `conv2D` 与池化; 构建 CNN; 在与 MLP 相近设置下对比并讨论 CNN 更优原因。

2.2 实现与 shape 流水线

Part B 在 Part A 相同数据划分上实现二维卷积、最大池化与 `Model_CNN`，入口为 `codes/test_train_cnn.py`。卷积与池化位于 `codes/mynn/op.py`，模型位于 `codes/mynn/models.py`；优化器、学习率调度与多分类交叉熵与 Part A 相同（§1.2 (3)(5)(6)），此处不重复公式。

张量形状 (batch 大小记为 N ；输入可为展平 $(N, 784)$ ，模型内 reshape 为 NCHW)：

阶段	形状
输入 / reshape	$(N, 784) \rightarrow (N, 1, 28, 28)$
<code>conv2D</code> (1→8, 核 3×3 , stride=1, pad=1)	$(N, 8, 28, 28)$
ReLU	$(N, 8, 28, 28)$
<code>MaxPool2D</code> (2×2 , stride=2)	$(N, 8, 14, 14)$
Flatten	$(N, 1568)$
Linear(1568→128) → ReLU	$(N, 128)$
Linear(128→10)	$(N, 10)$

可训练参数量 (粗算)： 卷积 $8 \times 1 \times 3 \times 3 + 8$ ，全连接 $1568 \times 128 + 128$ 、 $128 \times 10 + 10$ ，合计约 2.02×10^5 ，约为 Part A MLP ($\approx 4.77 \times 10^5$) 的 42%。

(1) 二维卷积 `conv2D` (im2col + 矩阵乘)

设输入 $X \in \mathbb{R}^{N \times C_{\text{in}} \times H \times W}$ ，卷积核张量 $W_c \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$ 。im2col 将每个感受野展成一行，得到 $\text{col} \in \mathbb{R}^{(N \cdot H_{\text{out}} W_{\text{out}}) \times (C_{\text{in}} k_h k_w)}$ ，再作

$$Y_{\text{flat}} = \text{col } W_{\text{mat}}^T + b, \quad W_{\text{mat}} = \text{reshape}(W_c)$$

输出 reshape 为 $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ 。本实验

$C_{\text{in}} = 1, C_{\text{out}} = 8, k = 3, \text{padding} = 1$, 故 $H_{\text{out}} = W_{\text{out}} = 28$, 在保持空间分辨率的同时在 3×3 邻域内聚合局部灰度模式。反向传播对 col 的梯度用矩阵乘得到 $\partial \mathcal{L} / \partial W_c$, 对输入的梯度经 col2im 累加回 padded 输入（重叠感受野处梯度相加）。

Python 实现:

```

1 class conv2D(Layer):
2     """
3     2D 卷积: im2col 展开感受野后矩阵乘; 反向 col2im 写回 dX。
4     W: [out_channels, in_channels, k_h, k_w]
5     """
6     def forward(self, X):
7         self.input = X
8         batch_size = X.shape[0]
9         col, out_h, out_w, X_padded = _im2col(X, self.kernel_size,
10 self.stride, self.padding)
11         self.col, self.out_h, self.out_w = col, out_h, out_w
12         W_mat = self.W.reshape(self.out_channels, -1)
13         out = col @ W_mat.T + self.b.reshape(1, -1)
14         return out.reshape(out_h, out_w, batch_size,
15 self.out_channels).transpose(2, 3, 0, 1)
16
17     def backward(self, grads):
18         # ——  $\partial \mathcal{L} / \partial W$ ,  $\partial \mathcal{L} / \partial b$ ; dcol = grads @ W_mat; col2im  $\rightarrow$  dX
19         ...

```

(2) 最大池化 MaxPool2D

对每一通道、每一个 $k_h \times k_w$ 窗口取最大值, 输出尺寸

$$H_{\text{out}} = \lfloor (H - k_h) / s_h \rfloor + 1, \quad W_{\text{out}} = \lfloor (W - k_w) / s_w \rfloor + 1$$

本层 $k = 2, s = 2$, 将 28×28 特征图降为 14×14 , 在保留强响应的同时降低全连接层输入维数。反向仅向每个窗口内 **argmax** 位置回传梯度, 其余位置为 0。

Python 实现:

```
1 class MaxPool2D(Layer):
2     def forward(self, X):
3         # —— 逐窗口 argmax, 记录 argmax_idx 供 backward ——
4         ...
5         return output
6
7     def backward(self, grads):
8         dX = np.zeros_like(self.input)
9         # —— 仅向 argmax 位置累加 grads ——
10        ...
11        return dX
```

(3) 模型 Model_CNN

Model_CNN 将 conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPool2D 与两层 Linear 串联: forward 先将 $(N, 784)$ reshape 为 $(N, 1, 28, 28)$, 卷积段结束后 flatten 为 1568 维, 再经 128 维隐藏层与 10 类 logits。backward 在 flatten 处先 reshape 回卷积输出形状, 再逆序穿过池化与卷积。卷积层与第一层全连接设 L2, $\lambda_{\text{conv}} = \lambda_{\text{fc}} = 10^{-4}$ (test_train_cnn.py 中 lambda_conv=1e-4, lambda_fc=1e-4)。

Python 实现:

```
1 class Model_CNN(Layer):
2     """
3     Conv(1->8, 3x3, pad=1) -> ReLU -> MaxPool(2x2)
4     -> Flatten(1568) -> Linear(1568->128) -> ReLU -> Linear(128->10)
5     """
```

```

6         def forward(self, X):
7             out = self._to_nchw(X)    # (N, 784) -> (N, 1, 28, 28)
8             for layer in self.layers[:3]:
9                 out = layer(out)
10            self._flatten_shape = out.shape
11            out = out.reshape(out.shape[0], -1)
12            for layer in self.layers[3:]:
13                out = layer(out)
14            return out
15
16        def backward(self, loss_grad):
17            # —— 先反传 FC，再 reshape 反传 conv 段 ——
18            ...

```

2.3 实验结果

实验设置与公平对比： 复用 Part A 生成的 `codes/idx.pickle` (`seed=309`, 50,000 train / 10,000 val) 与相同归一化；`codes/test_train_cnn.py` 使用 **SGD** (`init_lr=0.06`)、**MultiStepLR** (里程碑 [800, 2400, 4000], $\gamma = 0.5$)、`batch_size=32`、`num_epochs=5`、L2 $\lambda = 10^{-4}$ ，细节见 §1.2 (5)(6)。与 Part A 的**唯一训练流程差异**：`eval_each_iter=False`，即**每个 epoch 末**在验证集上评估并更新 best (Part A 为每个 batch 评估；`log_iters=100` 仅控制日志打印)。因 CNN 在 NumPy 上较慢，该设置减轻开销，主要影响 best checkpoint 选取粒度，不改变「CNN 明显优于 MLP」的结论 (见 §6.6)。权重保存至 `codes/best_models_cnn/best_model.pickle`；测试见 `codes/results/part_b_test_accuracy.txt`。

与 Part A 对比：

模型	最佳 val	测试	Checkpoint	验证频率
MLP (A)	95.04%	95.20%	<code>best_models/best_model.pickle</code>	每 batch

模型	最佳 val	测试	Checkpoint	验证 频率
CNN- SGD (B)	97.90%	97.98%	<code>best_models_cnn/best_model.pickle</code>	每 epoch 末

CNN-SGD 相对 MLP, test 提升约 2.78 个百分点。Part B 末轮训练准确率 96.88%、验证 97.90% (`part_b_summary.txt`)。

来源: `codes/results/part_b_summary.txt`、
`codes/results/part_b_test_accuracy.txt`。

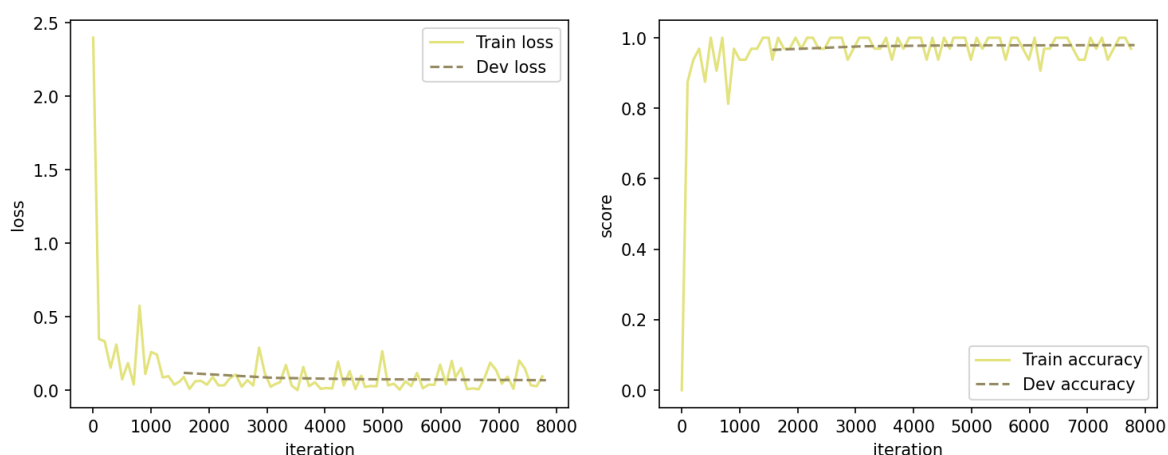


图2: CNN-SGD 学习曲线 (`codes/results/part_b_learning_curve.png`)。因 `eval_each_iter=False`, 验证曲线为每 epoch 一个 dev 点 (共 5 点), 呈阶梯上升; epoch 0 末 dev 约 96.55% (`codes/log/test_train_part_b.log`), 末 epoch 达 best val 97.90%。训练 batch 曲线仍逐 iter 记录, 末轮 train accuracy 96.88%, 低于 Part A 末轮 train 100%, 验证—训练间隙小于 MLP, 泛化更稳。

2.4 结果小结

在相同 `idx.pickle` 与对齐的超参 (epoch、batch、学习率调度、L2) 下, CNN-SGD 的 val/test 均显著高于 MLP (见上文表与 §4 总表)。CNN 以更少参数取得更高精度, 提示增益主要来自卷积+池化的局部性与平移归纳偏置; 机制讨论与困难样本分析见 §6。

3. Two Additional Directions

控制变量： MultiStepLR 与 L2 **已在 Part B 使用**；C-1 **仅**将优化器由 SGD 换为 **MomentGD ($\mu=0.9$)**，其余（结构、`idx.pickle`、5 epoch、batch 32、lr、调度、`eval_each_iter=False`）与 Part B 相同。

3.1 Direction 1: Optimization

3.1.1 原理与实现

Part C 方向一在 Part B 的 CNN 与训练配置（`idx.pickle`、batch、epoch、L2、MultiStepLR 等）基础上，将优化器换为 `MomentGD` ($\mu = 0.9$)。实现位于 `codes/mynn/optimizer.py` 与 `codes/mynn/lr_scheduler.py`；训练入口为 `codes/test_train_part_c_momentum.py`。

(1) 动量法

记深度神经网络为 $f(x; \theta)$ ，损失函数为 $\mathcal{L}(y, f(x; \theta))$ ，其中 θ 为网络参数。记第 k 步在 mini-batch $\mathcal{S}_k$ 上（`batch_size=32`）的梯度为 $g^{(k)}$ （反向传播写入各层 `layer.grads`），第 k 步学习率为 α_k （由 (2) 的 `MultiStepLR` 写入 `optimizer.init_lr`）。

动量法（momentum method）用之前积累的动量来修正更新方向，而不是仅用当前梯度；每次迭代的梯度可看作加速度。在第 k 次迭代时，计算负梯度的「加权移动平均」作为参数的更新方向：

$$\Delta\theta^{(k)} := \mu \Delta\theta^{(k-1)} - \alpha_k g^{(k)} = - \sum_{t=1}^k \alpha_t \mu^{k-t} g^{(t)}$$

其中动量因子 μ 取 0.9。参数更新为 $\theta^{(k)} \leftarrow \theta^{(k-1)} + \Delta\theta^{(k)}$ ；代码中记 $\Delta\theta$ 为 `self._velocity[...]`。

每个参数的实际更新差值取决于最近一段时间内梯度的加权平均值：梯度方向不一致时，真实更新幅度变小；方向一致时，更新幅度变大，起到加速作用。一般而言，迭代初期梯度方向较一致，动量法可更快下降 loss；迭代后期在收敛值附近梯度方向多变，动量法有助于抑制震荡。学习率 α_k 变小时，动量状态 `_velocity` **不**随 milestone 清零，仍参与后续更新。

Python 实现:

```

1 class MomentGD(Optimizer):
2     """
3     带动量的随机梯度下降（MomentGD）。
4
5     参数
6     ----
7     init_lr : float
8         当前学习率  $\alpha_k$ （由 MultiStepLR 在 milestone 处缩放）
9     model : 模型实例
10         需有 model.layers，每层提供 .params、.grads、.optimizable
11     mu : float, default=0.9
12         动量因子  $\mu$ 
13     """
14     def __init__(self, init_lr, model, mu=0.9):
15         super().__init__(init_lr, model)
16         self.mu = mu
17         self._velocity = {} # 每层每个参数一个速度，懒分配
18
19     def step(self):
20         for layer in self.model.layers:
21             if layer.optimizable:
22                 for key in layer.params.keys():
23                     if layer.weight_decay:
24                         layer.params[key] *= (
25                             1 - self.init_lr *
26                             layer.weight_decay_lambda
27                             )
28                     vkey = (id(layer), key)
29                     if vkey not in self._velocity:
30                         self._velocity[vkey] =
31                             np.zeros_like(layer.params[key])
32                     # ——  $v_t = \mu v_{t-1} - \alpha_k g^{\{(k)\}}$ 
33
34                     self._velocity[vkey] = (
35                         self.mu * self._velocity[vkey]
36                         - self.init_lr * layer.grads[key]
37                     )

```

```

34         )
35         #  $\theta \leftarrow \theta + \Delta \theta_k$ 
36         layer.params[key] += self._velocity[vkey]

```

(2) 多步里程碑衰减 (MultiStepLR)

学习率在一开始宜较大以加快收敛，接近最优点时宜减小以避免振荡。本实验采用**学习率衰减**：在 `lr_scheduler.py` 中按每次迭代更新 `optimizer.init_lr`，记第 k 步学习率为 α_k ；`MomentGD` 在 `step()` 内用该值计算梯度项与 L2 系数。`RunnerM` 在每个 batch 内顺序为：前向与反传 $\rightarrow$ `optimizer.step()` $\rightarrow$ `scheduler.step()`。

在预设迭代步数 **milestones** 处，将当前学习率乘以 $\gamma \in (0, 1)$ 。记 milestone 集合为 $\mathcal{M}$ ，则：

$$\alpha_k := \begin{cases} \gamma \alpha_{k-1}, & k \in \mathcal{M} \\ \alpha_{k-1}, & \text{otherwise} \end{cases} \quad \alpha_0 = 0.06$$

本实验 $\mathcal{M} = \{800, 2400, 4000\}$ ， $\gamma = 0.5$ ，故 α_k 序列为 $0.06 \rightarrow 0.03 \rightarrow 0.015 \rightarrow 0.0075$ ($k \geq 4000$ 后保持不变)。调度器**只修改** `optimizer.init_lr`，**不重置** `MomentGD._velocity`。

Python 实现:

```

1 class MultiStepLR(scheduler):
2     def __init__(self, optimizer, milestones, gamma=0.1):
3         """
4         milestones : list of int
5             达到这些 step_count 时，将 optimizer.init_lr 乘以
6             gamma
7         gamma : float
8             学习率缩放系数，0 < gamma < 1
9         """
10        super().__init__(optimizer)
11        self.milestones = milestones
12        self.gamma = gamma
13
14    def step(self) -> None:

```

```

14         self.step_count += 1
15         if self.step_count in self.milestones:
16             self.optimizer.init_lr *= self.gamma

```

训练脚本中的实例化：`MultiStepLR(optimizer=optimizer, milestones=[800, 2400, 4000], gamma=0.5)`。

3.1.2 对比结果

指标	CNN-SGD (B)	CNN-MomentGD (C)
最佳 val	97.90%	98.86%
测试	97.98%	98.74%

逐 epoch dev:

Epoch	SGD	MomentGD
0	96.55%	97.48%
1	97.57%	98.50%
2	97.83%	98.72%
3	97.84%	98.85%
4	97.90%	98.86%

来源: `results/part_c_momentum_summary.txt` (SGD 逐 epoch 为同文件 baseline 行, 来自 `training_part_b.log`)。

训练准确率对比: Part B SGD 末 epoch 训练准确率 **96.88%**

(`part_b_summary.txt`) ; Part C MomentGD 末 epoch **100.00%**

(`part_c_momentum_summary.txt`)。MomentGD 在验证集上更早、更高 (epoch 0 即 97.48% vs 96.55%) , 同时末轮训练拟合更强, dev/test 仍优于 SGD。

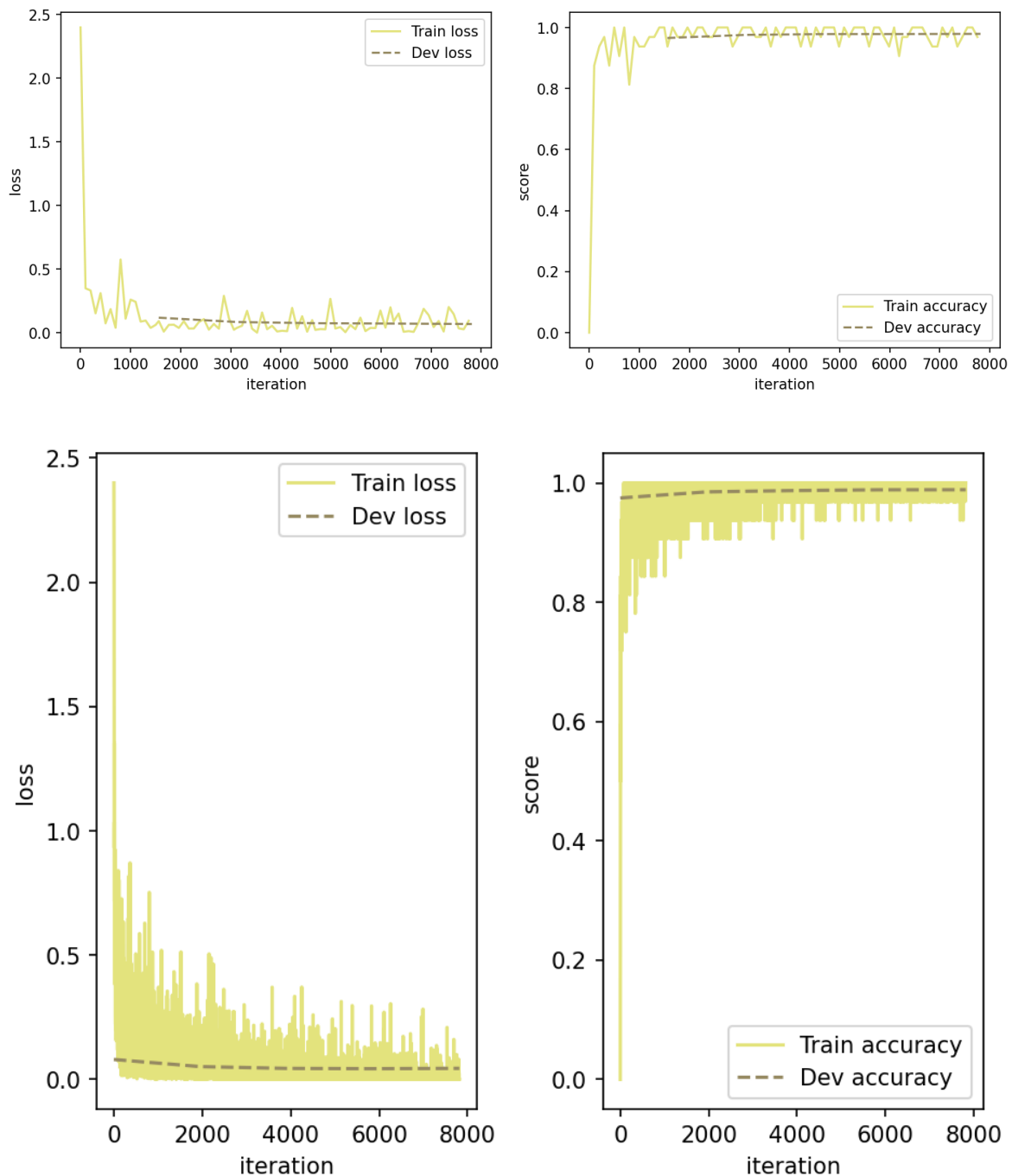
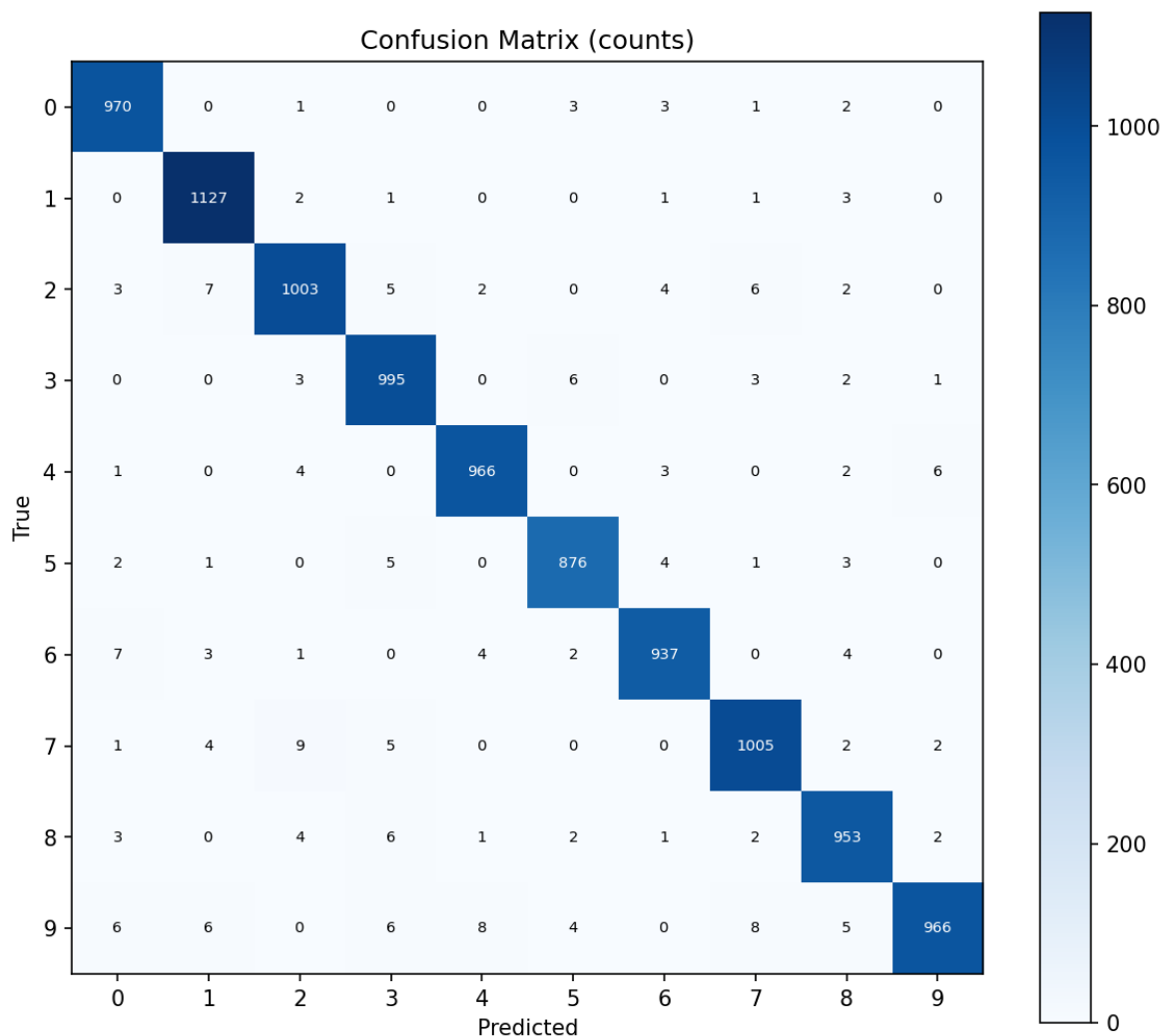


图 3-4: 优化器对照。MomentGD 曲线显示 dev 在前 1~2 epoch 领先约 0.8~1.0 个百分点, 最终 test 高 0.76 个百分点。

3.2 Direction 2: Error Analysis and Visualization

(1) Confusion matrix

在 10×10 混淆矩阵上统计预测结果



(图: `codes/results/part_c/part_c_confusion_matrix.png`)

对角线占主导，说明多数样本分类正确；off-diagonal 中非零主要集中在少数格子，其中 7→2 (9)、9→7 (8)、9→4 (8) 次数最高。错误非均匀分布，而是集中在形态相近的数字对。

**

true → pred

次数

true → pred	次数
7 → 2	9
9 → 7	8
9 → 4	8
6 → 0	7
2 → 1	7

(2) Misclassified examples

从测试集错分样本中选取 16 张展示

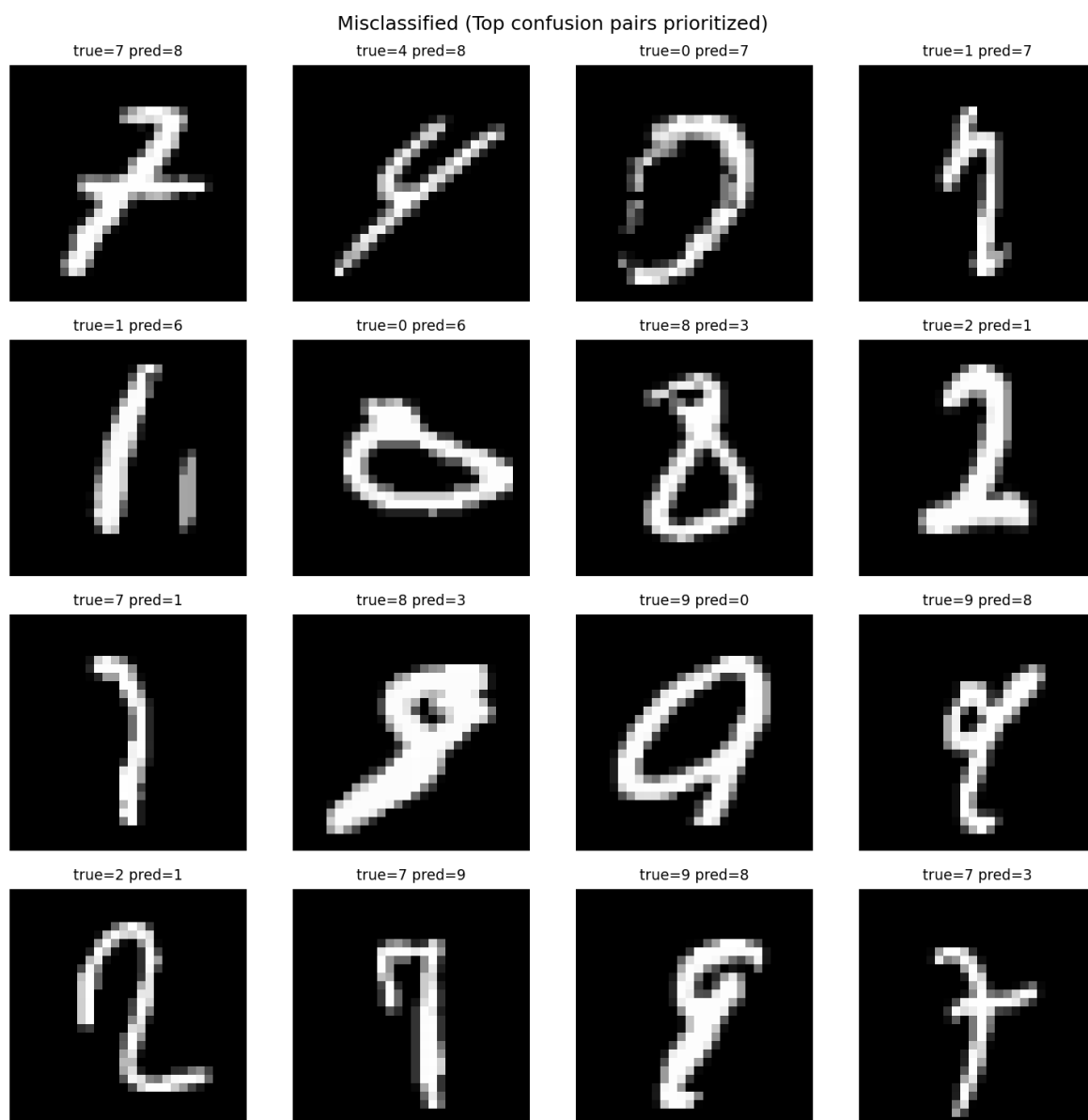
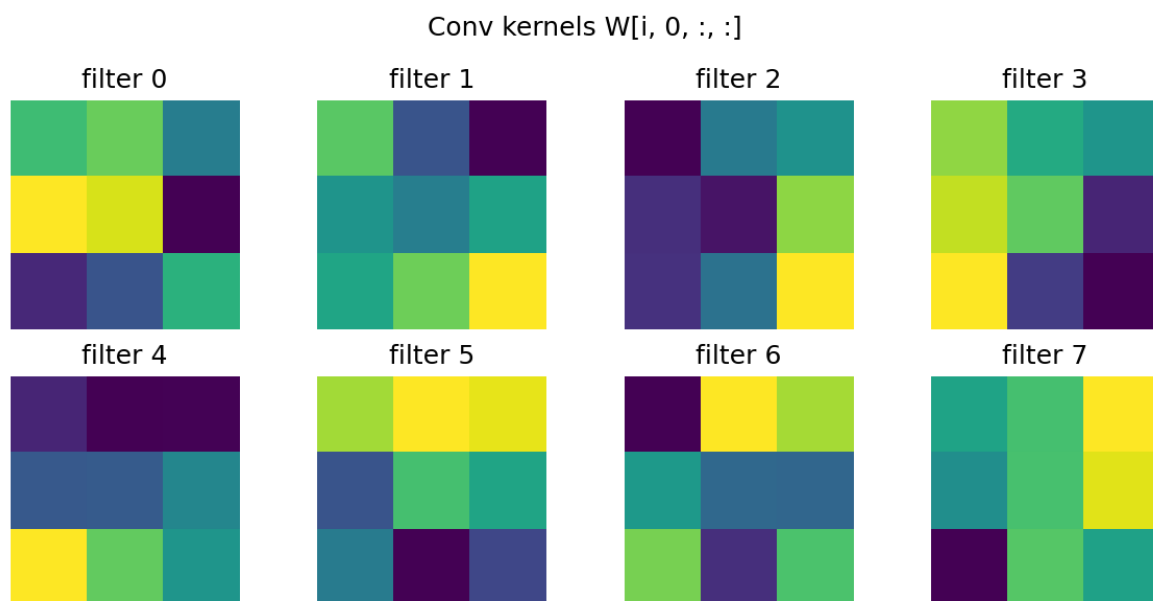


图 `part_c_misclassified.png` 展示 16 张测试集错例 (`seed=309` , 优先从高频混淆对对应错例中抽样, 但 16 张未覆盖全部 Top 对)。可见标签包括: 7→8、7→1、7→9、7→3; 9→0、9→8; 以及2→1、8→3、0→7等。例如true=7, pred=9的样本中, 竖笔与顶部弧线在局部窗口内与「9」的上结构相近; true=9, pred=8时尾部回卷使轮廓接近「8」。

结论: 错例图像说明错误多来自**笔画形态变异**、**环闭合与否**、**竖笔/横折**等局部相似, 属于类间视觉混淆, 而非随机噪声。

(3) Visualization of convolution kernels

对 Part B 第一层卷积权重可视化，取 `W[i, 0, :, :]` (8 个 3×3 滤波器)：



部分核呈沿某一方向的亮暗变化，对应近似边缘/角点响应，说明网络在学习局部笔画特征；与混淆矩阵、错分图结合看，高层仍会在相似数字对之间混淆。

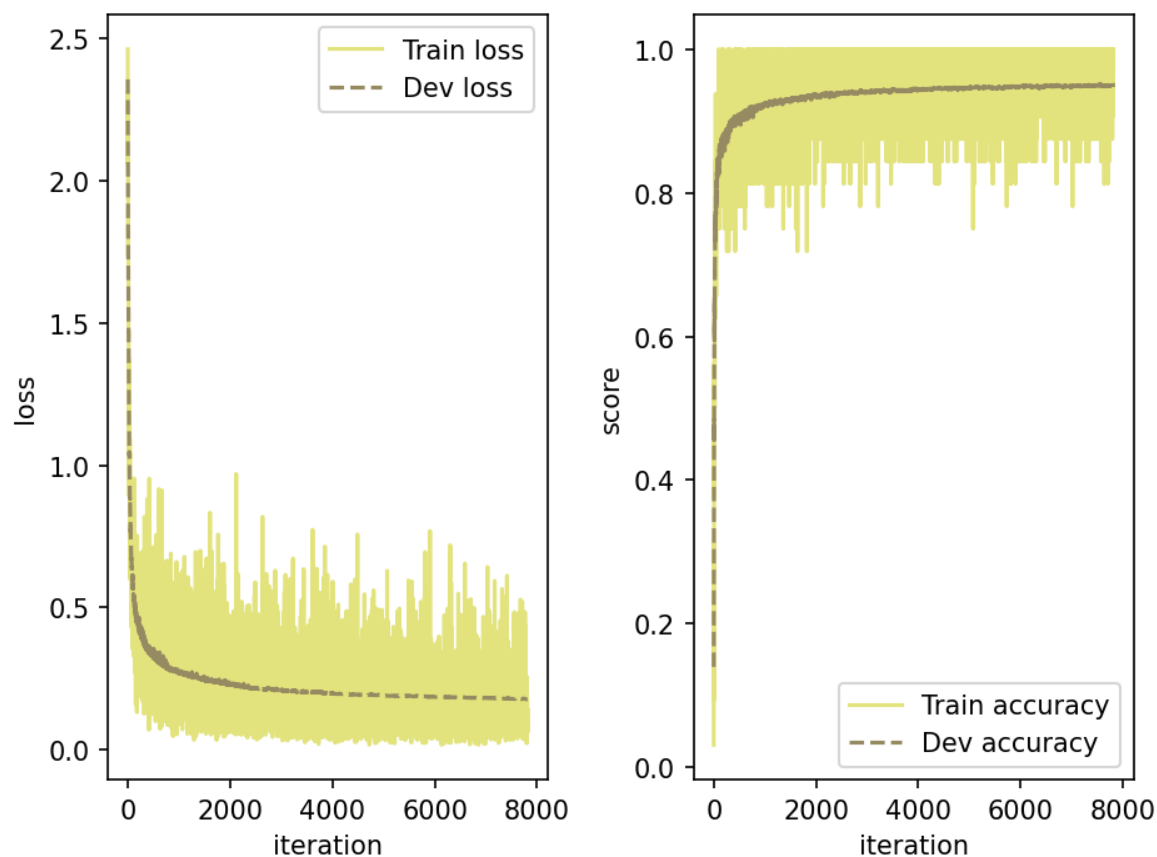
三项分析相互印证：混淆矩阵给出哪些类易混，错分图给出形态原因，卷积核说明底层学到了什么；共同解释在 test 97.98% 时仍困难的主要是 9↔4/7、7↔2 等样本。

4. Main Results Table

部分	模型	优化器	最佳 val	测试	Checkpoint	备注
A	MLP 784→600→10	SGD	95.04%	95.20%	<code>best_models/best_model.pickle</code>	eval 每 100 iter
B	CNN (§2.2)	SGD	97.90%	97.98%	<code>best_models_cnn/best_model.pickle</code>	同 idx
C-1	同 B	MomentGD $\mu=0.9$	98.86%	98.74%	<code>best_models_cnn_momentum/</code>	仅换优化器
C-2	同 B (分析)	SGD	97.90%	97.98%	仅用 B 权重	错分 202; 非 MomentGD

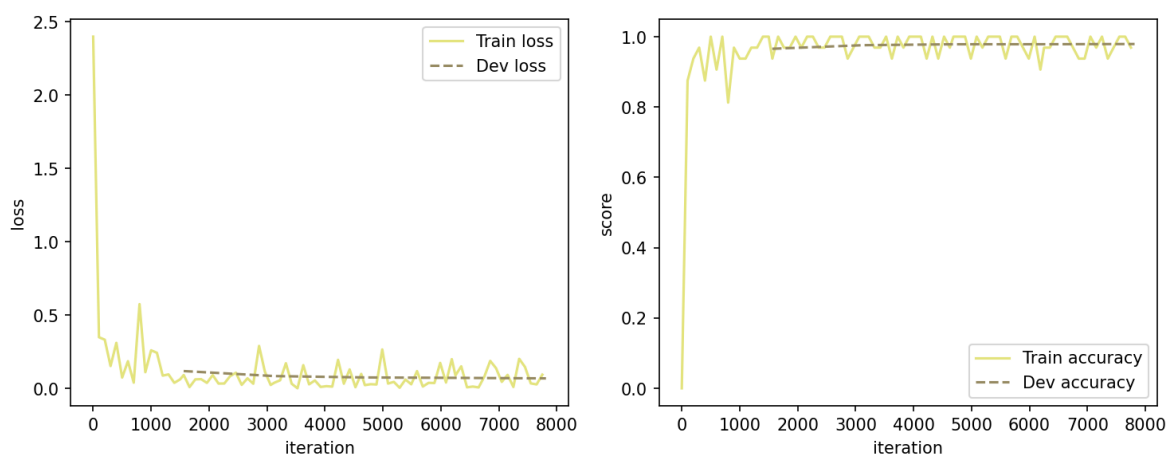
5. Detailed Visualization

5.1 Part A — MLP 学习曲线



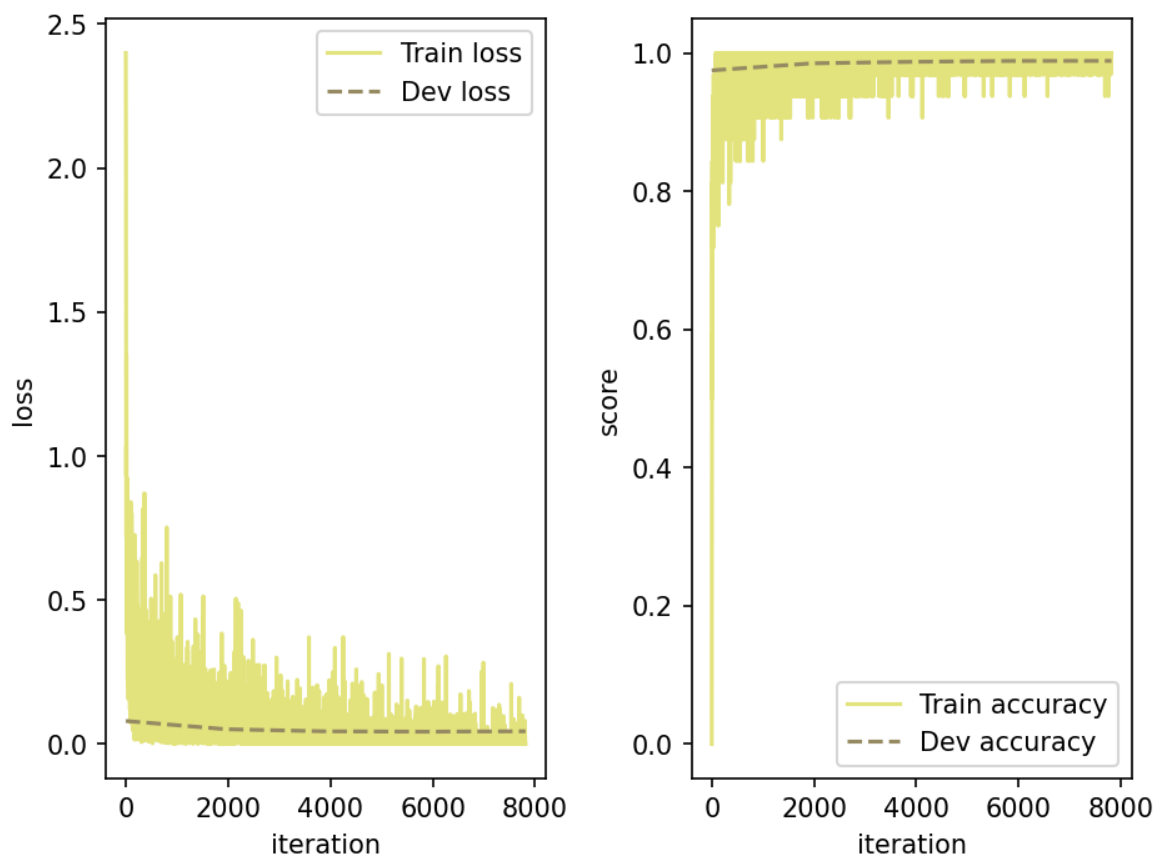
训练 loss 单调下降；验证 accuracy 在约 7000 iteration 内升至 95% 平台。曲线中 train 与 dev 的差距反映 MLP 对训练 batch 的更强拟合，与 §1 过拟合描述一致。

5.2 Part B — CNN-SGD 学习曲线



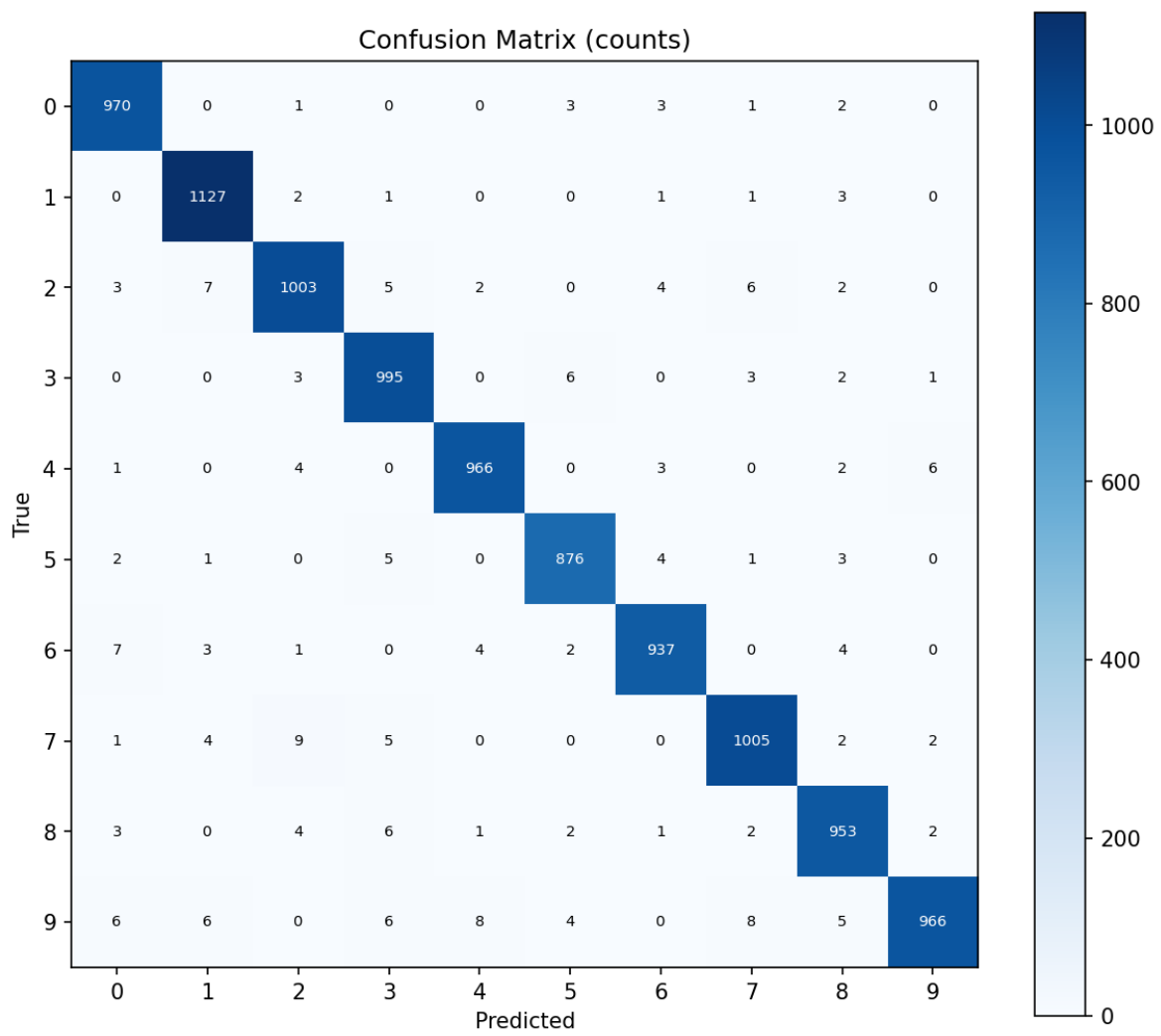
每个 epoch 末 dev 点呈阶梯上升，5 个 epoch 内从约 96.6% 升至 97.9%。相比图 5.1，同量级 iteration 下 dev 明显高于 MLP，体现卷积+池化对手写数字的有效性。

5.3 Part C — MomentGD 学习曲线



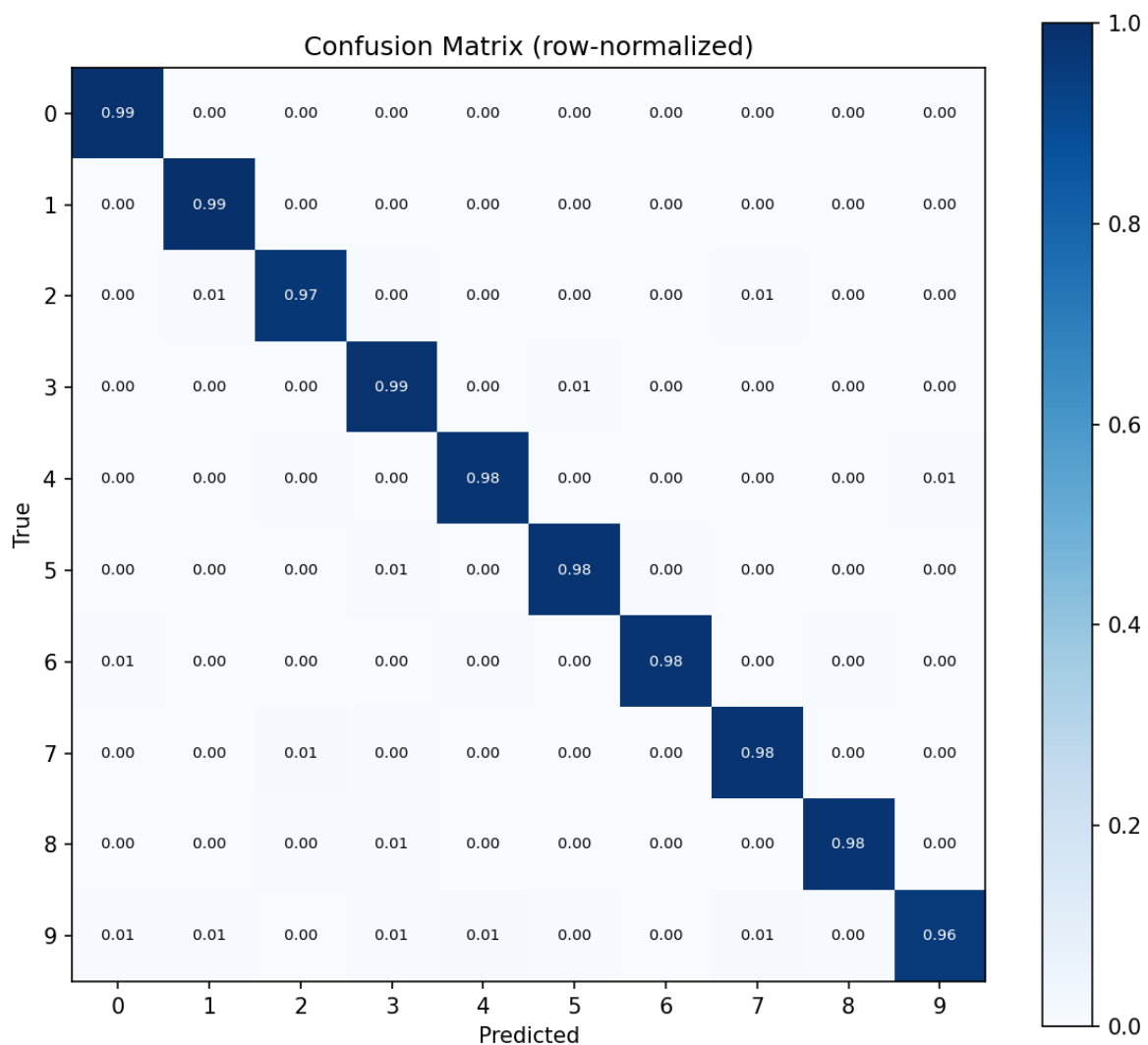
与图 5.2 对照：相同结构与超参下，MomentGD 的 dev 曲线整体更高、更陡，尤其前 1~2 epoch；与 §3.1 逐 epoch 表一致，支持「动量加速早期收敛」的判断。

5.4 混淆矩阵（计数）



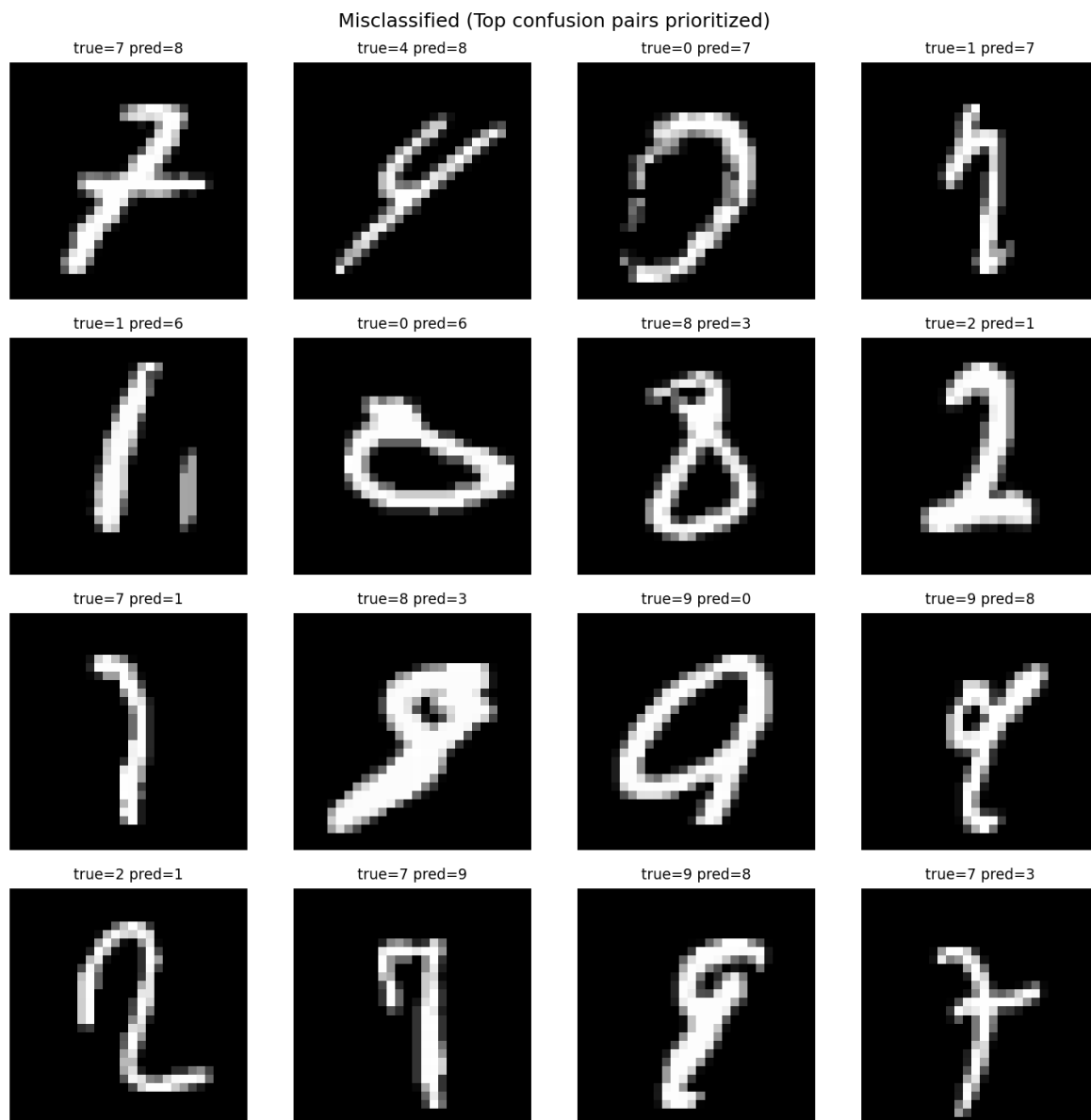
对角线为主（正确分类）；非对角块显示 7→2、9→7、9→4 等 off-diagonal 计数最大，与 Top 混淆表一致。多数类 recall 高于 97%，错误集中在少数形态相近数字对。

5.5 混淆矩阵（行归一化）



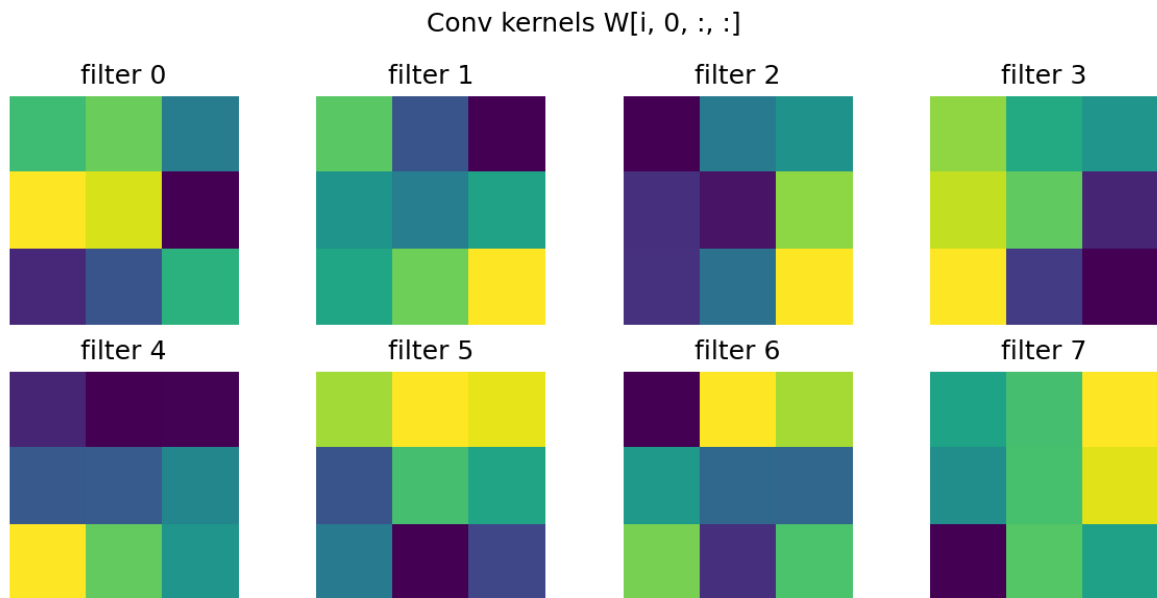
按真实类行归一化后，**第 9 行**非对角概率相对突出（漏检 9 被预测为 4/7 等），与 recall 0.9574 一致；**第 7 行**对列 2 有一定 off-diagonal 质量，对应 7→2。该图便于比较「某类被误分到哪」而非绝对计数。

5.6 错分样本



16 张错例优先从 Top 混淆对（如 7→2、9→4/7）抽样（`seed=309`）。可见 7 与 2 笔画拓扑相近（7→2）；9 的上环与 4/7 在细笔画、倾斜书写下易混（9→4、9→7）。非均匀随机采样，便于与混淆矩阵对照阅读。

5.7 卷积核



8 个滤波器 $W[i, 0, :, :]$ (3×3)。部分核呈近似 **垂直/水平边缘** 或 **角点** 响应（亮暗对比沿某一方向变化），符合第一层卷积学习局部梯度模板的预期；各核模式不完全相同，提供多样局部特征供后续全连接组合。

6. Discussion

6.1 Why is CNN more suitable than MLP for image classification?

MNIST 像素在二维网格上具有**强局部相关性**：相邻像素往往同属一笔画，而相距较远的像素相关性弱。手写数字还具有近似**平移不变性**，笔画模式出现在不同位置时，类别不应改变。MLP 将 28×28 图像**展平**为 784 维向量后，第一层即为 784×600 的全连接映射（参数量约 4.77×10^5 ），每个输出单元与**全部像素**相连，既未显式利用邻域，也需在大量连接中自行“发现”局部模式，归纳偏置与图像几何不匹配。

学习曲线 (§5.1、§5.2) 与上述判断相符：MLP 验证准确率在前 1~2 个 epoch 内快速升至约 95% 后进入平台；CNN-SGD 在 5 个 epoch 内 dev 由约 96.6% (epoch 0 末，见 `training_part_b.log`) 稳步升至 97.90%。在相近训练预算下，CNN 的 dev 曲线整体高于 MLP，更适合网格图像。

6.2 Does the CNN improve validation or test accuracy?

是。在相同训练/验证划分 (`idx.pickle`, `seed=309`) 及对齐的训练超参 (5 epoch、batch 32、`init_lr=0.06`、MultiStepLR、L2) 下, CNN-SGD 相对 MLP 在验证集与测试集上均有显著提升。数值如下 (来源: `results/part_a_summary.txt`、`part_b_summary.txt` 及对应 `*_test_accuracy.txt`) :

模型	最佳验证准确率	测试准确率	相对 MLP 提升 (test)
MLP (Part A)	95.04%	95.20%	—
CNN-SGD (Part B)	97.90%	97.98%	+2.78 pp

验证集上 CNN 比 MLP 高约 2.86 pp (97.90% vs 95.04%) , 测试集上高约 2.78 pp。Part B 末 epoch 训练准确率 96.88%、验证 97.90% (`part_b_summary.txt`) , 而 Part A MLP 末轮训练 batch 准确率可达 100%、验证 95.04% (`part_a_summary.txt`) , 表明 CNN 的泛化间隙相对 MLP 更小, 过拟合程度更轻。

6.3 Which two additional directions did you choose, and why?

本实验选择: Optimization; Error Analysis and Visualization

6.3.1 Optimization

Part B 已在 MNIST 上得到稳定的 CNN-SGD 基线 (val 97.90%, test 97.98%) 。在此基础上, 若同时改动网络深度、数据增广或学习率策略, 难以判断性能变化来自哪一因素。因此 C-1 仅将优化器替换为带动量的 MomentGD ($\mu=0.9$) , 其余结构、数据划分、`idx.pickle`、5 epoch、batch、初始学习率、MultiStepLR 与 L2 均

与 Part B 相同。该设计可在控制变量前提下回答：在已有 CNN 上，动量是否加快收敛并提高最终 val/test。

6.3.2 Error Analysis and Visualization

当 test 已达 97.98%** 时，剩余错误虽仅占 2.02% (202/10000)，但对理解模型行为仍有价值。本方向在**固定 Part B SGD 权重**

(`best_models_cnn/best_model.pickle`) 上，对测试集做混淆矩阵（计数/行归一化）、Top 混淆对统计、错分样本网格与第一层卷积核可视化 (§3.2、§5.4–5.7)，**不使用** MomentGD 权重，以免将“优化带来的精度提升”与“SGD 模型下的失败模式”混为一谈。

6.3.3 未选其他方向的原因：

更深 CNN、数据增广、Dropout/BN 等在本作业框架（纯 NumPy、CPU、单次训练约数十分钟）下会显著增加实现与调参成本；相较之下，换优化器与误差分析分别覆盖“训练动力学”与“可解释性”两类问题，且与课程提供的 `MomentGD` 实现任务、报告 Appendix A.6 的讨论题直接对应，

6.4 Which modification or analysis is the most informative?

Part C 方向2（以下简称C-2） Error Analysis and Visualization

方向一（MomentGD）在控制变量下将 test 从 97.98% 提到 98.74% (+0.76 pp)，epoch 0 dev 由 96.55% 升至 97.48%，符合「动量加速早期收敛」的常见预期，属于性能层面的增量改进，对「模型还在哪些样本上失败、为何失败」揭示有限。当 CNN-SGD 已在 test 97.98% 时，准确率数字本身难以说明剩余 202 张错分 (2.02%) 的结构。

C-2 固定 `best_models_cnn/best_model.pickle` (与 `part_c_error_analysis.txt` 一致)，从三条线索形成可检验的解释链：

1. **混淆矩阵** (§5.4–5.5)：错误非均匀散布，Top 对为 7→2 (9)、9→7 (8)、9→4 (8)；行归一化显示类 9 recall 最低 (0.9574)，指出薄弱类别而非笼统「模型不好」。

- 2. 错分样本图 (§5.6) : 16 张错例呈现笔画闭合、竖笔/横折、上环不完整等形态机制，把矩阵中的计数对应到可观察图像。
- 3. 卷积核可视化 (§5.7) : 第一层 8 个 3×3 核呈边缘/角点类响应，说明底层已学局部特征，与「高层仍在 7/9 等相似类之间混淆」并不矛盾。

三者相互印证：高准确率下的残差主要是类间视觉混淆，而非实现错误或特征完全未学习。这对后续改进（针对类 9、难例增广等）比单纯再提高 0.76 pp 更具诊断价值。

只看MomentGD，易得出换优化器的结论，但不知道错误模式。误差分析在不改动训练的前提下回答了错在哪里、长什么样的问题，因此最具启发。

6.5 What kinds of samples are still hard for your model?

以下分析仅针对 Part B CNN-SGD 在测试集上的表现

(`best_models_cnn/best_model.pickle` , test acc 97.98%, 与 `part_c_error_analysis.txt` 一致) , 不包含 MomentGD 模型上的错分。

整体

共 10,000 张测试样本，错分 202 张，错分率 2.02%。在 per-class 指标中，类 9 的 recall 最低 (0.9574) , 其次类 2 (recall 0.9719) 、类 7 (0.9776) ; 多数类别 recall 在 97.8% ~ 99.3% 之间，说明困难样本集中在少数类别及其混淆对上。

最主要的类别间混淆 (true → pred, 次数)

true → pred	次数	简要形态解释
7 → 2	9	7 的横笔与斜笔组合在局部窗口内可与 2 的弧度笔画相似
9 → 7	8	9 的竖笔与上环在书写潦草时易被读成 7
9 → 4	8	9 闭合环与 4 的开口结构在细笔画 倾斜时相近

true → pred ⁰	次数	简要形态解释 6 的下环与 0 的椭圆局部相似
2 → 1	7	2 与 1 在竖笔主导的写法下拓扑接近

上述五对占 Top 列表 (`part_c_error_analysis.txt`) , 与计数版/行归一化混淆矩阵 (§5.4、§5.5) 中非对角块亮区一致。

按类看的薄弱点

- 类 9: precision 0.9887 高、recall 0.9574 最低, 典型模式是**真 9 被误判为 4 或 7** (见上表) , 符合“上环 + 竖笔”与 4/7 共享局部结构的现象。
- 类 2: recall 0.9719, 除 2→1 外, 在 Top 对中亦有与其他类混淆的个案。
- 类 7: recall 0.9776, 与 7→2 的 Top 计数呼应。

与可视化的对应

`part_c_misclassified.png` (§5.6) 优先从上述 Top 混淆对中抽样 (`seed=309`) , 可见非随机错分: 例如 **true=7, pred=2** 的样本中横折笔画占据主导; **true=9, pred=4/7** 的样本中上环不完整或倾斜导致与 4/7 局部匹配。这些个案说明: 在 98% 附近的准确率下, 剩余错误主要来自**类间视觉相似与书写变体**, 而非大规模未学习 (第一层卷积核仍呈现边缘/梯度模式, §5.7) 。

未解决的问题

本实验未对 202 张错分做人工逐张标注或增广重训; 针对类 9 的 recall 提升可能需要更强增广、更深网络或难例挖掘, 见 §6.6。

6.6 局限与改进

实验在纯 NumPy + CPU 下完成, 卷积与逐 epoch 全量验证耗时较长; 网络为单卷积层 8 通道 + 两层全连接、5 epoch、无数据增广, 未探索更深 CNN、Dropout/BatchNorm 等。MomentGD 末轮 train 100% 提示仍存在一定过拟合风险, 但 test 仍优于 SGD。针对类 9 等低 recall 类别, 后续可从增广、难例挖掘或更深结构等方向扩展——本次作业选择动量优化与误差可视化, 是为在有限实现成本下分别覆盖「训练动力学」与「模型可解释性」两类问题。

